

Guatemala, 20 de septiembre de 2026

Universidad del Valle de Guatemala

Ingeniería en Ciencias de la Computación y Tecnologías de la Información

CC3067 Redes



Proyecto 1
Uso de un protocolo existente
Reporte final

Melisa Mendizabal (23778)

Enlace al Repositorio: https://github.com/melisadmendizabal/redes_proyecto1

1. Alcance de esta entrega

Este documento es el reporte final del proyecto y sustituye al reporte parcial de la Entrega 1. Cubre los incisos de reporte solicitados en el enunciado: el inciso 8 (especificación, parámetros y endpoints de los servidores MCP desarrollados), el inciso 9 (explicación, a partir del análisis con Wireshark, de lo que sucede en las capas de enlace, red, transporte y aplicación) y el inciso 10 (conclusiones y comentarios sobre el proyecto). Además, documenta la evidencia del inciso 7 (clasificación de los mensajes JSON-RPC capturados entre el anfitrión y el servidor remoto) y la interfaz de usuario web implementada como extra.

El chatbot (anfitrión), implementado en Python, se conecta a tres servidores MCP mediante un cliente propio, sin usar SDKs de MCP, hablando JSON-RPC 2.0 de forma manual. Según el servidor, el transporte es local (stdio) o remoto (Streamable HTTP):

- Filesystem MCP Server (oficial, @modelcontextprotocol/server-filesystem): local, stdio.
- Git MCP Server (oficial, mcp-server-git): local, stdio.
- Pharmacy MCP Server (propio, implementado desde cero): la misma lógica de negocio expuesta por dos transportes, local (stdio) y remoto (Streamable HTTP, publicado en Render).

Resumen de funcionalidades frente al enunciado:

#	Funcionalidad	Estado	Dónde se documenta
1	Conexión con un LLM a nivel de API (Google Gemini)	Implementada	core/llm_client.py; sección 1.2
2	Contexto de sesión	Implementada	core/chatbot.py; sección 2.3 y Figura 2
3	Log de solicitudes y respuestas MCP	Implementada	core/logger.py; comando /log y botón "Log MCP" de la interfaz web
4	Servidores oficiales locales: Filesystem y Git	Implementada	Secciones 2.1 y 2.2
5	Servidor MCP propio, local (farmacia)	Implementada	Sección 2.3
6	Servidor MCP propio, remoto	Implementada (Render)	Sección 2.4
7	Captura y análisis con Wireshark	Realizada	Sección 4
8	Especificación de los servidores	Incluida	Sección 2
9	Análisis por capas (enlace, red, transporte, aplicación)	Incluido	Sección 4.4
10	Conclusiones y comentarios	Incluidas	Sección 5
Extra	Interfaz de usuario web	Implementada	Sección 3

1.1 Arquitectura general

El proyecto está organizado en capas, de modo que cada pieza puede cambiarse sin afectar a las demás:

- core/: lógica independiente de la interfaz. llm_client.py (API de Gemini), chatbot.py (historial por sesión y ciclo de function calling), mcp_client.py (clientes MCPStdioClient y MCPHttpClient), mcp_manager.py (registra los servidores, unifica sus herramientas y enruta cada llamada al servidor que la expone) y logger.py (registro de todos los mensajes JSON-RPC).
- interfaces/: capas delgadas, sin lógica de negocio: cli.py (consola) y web.py (API HTTP que consume la interfaz web).

- `servers/pharmacy/`: servidor propio. `logic.py` (catálogo y herramientas, compartido), `local_stdio.py` (transporte stdio) y `remote_http.py` (transporte HTTP).
- `main.py` y `main_web.py`: puntos de entrada; deciden qué servidores se levantan.

Los dos clientes MCP exponen la misma interfaz (`start`, `initialize`, `list_tools`, `call_tool`, `close`), por lo que el manager y el chatbot no distinguen si una herramienta vive en un subproceso local o en un servidor remoto. El modelo decide qué herramienta usar mediante `function calling`: el manager traduce el `inputSchema` de cada herramienta MCP al formato `functionDeclarations` de Gemini.

1.2 Nota sobre el proveedor del LLM

El enunciado del proyecto sugiere el uso de los LLMs de Anthropic, que ofrecen \$5 de crédito gratuito en su API. Sin embargo, no fue posible activar dicho crédito de prueba: la plataforma de Anthropic requiere verificación telefónica por SMS para habilitarlo, y esta verificación no fue aceptada para el número de teléfono correspondiente a la región (Guatemala). Por esta razón, se utilizó como proveedor del LLM la API de Google Gemini en su lugar.

Este cambio de proveedor no afectó la arquitectura general del proyecto: la comunicación con el LLM está encapsulada en un único módulo (`core/llm_client.py`), completamente independiente del cliente MCP, del manager de servidores y de la interfaz de consola. Cambiar de proveedor implicó únicamente adaptar ese módulo y el manejo del historial de conversación (`core/chatbot.py`), sin modificar el resto del sistema.

2. Especificación de los servidores MCP (inciso 8)

2.1 Filesystem MCP Server (oficial)

Servidor oficial de Anthropic, ejecutado localmente vía subproceso (`npx`). Expone operaciones de manipulación de archivos dentro de un directorio restringido (`allowed directory`), pasado como argumento al iniciar el servidor.

- Comando de arranque:

```
npx -y @modelcontextprotocol/server-filesystem <ruta_permitida>
```

- Herramientas relevantes utilizadas en la demo de este proyecto:

Herramienta	Parámetros	Descripción
<code>write_file</code>	<code>path</code> (string), <code>content</code> (string)	Crea o sobrescribe un archivo con el contenido indicado.
<code>list_allowed_directories</code>	(sin parámetros)	Lista los directorios donde el servidor tiene permitido operar.

- Ejemplo real de solicitud/respuesta JSON-RPC capturado durante la demo:

```
// Solicitud
{"jsonrpc": "2.0", "id": 4, "method": "tools/call", "params": {
  "name": "write_file", "arguments": {
    "content": "Proyecto MCP - CC3067",
    "path": "...\\workspace\\README.md"
  }}
}

// Respuesta
{"jsonrpc": "2.0", "id": 4, "result": {
  "content": [
    {
      "type": "text",
      "text": "Successfully wrote to ...\\workspace\\README.md"
    }
  ]
}}
```

2.2 Git MCP Server (oficial)

Servidor oficial de Anthropic (paquete de Python mcp-server-git), ejecutado localmente. Expone operaciones de control de versiones sobre un repositorio git indicado por su ruta absoluta (repo_path) en cada llamada.

- Comando de arranque:

```
python -m mcp_server_git --repository <ruta_del_repo>
```

- Herramientas relevantes utilizadas en la demo de este proyecto:

Herramienta	Parámetros	Descripción
git_status	repo_path (string)	Muestra el estado del árbol de trabajo del repositorio.
git_add	repo_path (string), files (array de strings)	Agrega archivos al área de staging.
git_commit	repo_path (string), message (string)	Registra un commit con los cambios en staging.

- Ejemplo real de solicitud/respuesta JSON-RPC capturado durante la demo:

```
// Solicitud
{"jsonrpc":"2.0","id":5,"method":"tools/call","params":
  {"name":"git_commit","arguments":{
    "repo_path":"...\\workspace","message":"initial commit"}}}

// Respuesta
{"jsonrpc":"2.0","id":5,"result":{"content":[{"type":"text",
  "text":"Changes committed successfully with hash 838e4ab6..."}],
  "isError":false}}
```

2.3 Pharmacy MCP Server (propio): versión local (stdio)

Servidor MCP implementado desde cero para este proyecto (sin SDK de MCP), correspondiente al caso de uso de industria: una cadena de farmacias que recomienda y vende medicamentos según los síntomas del cliente. Se ejecuta localmente vía stdio, hablando JSON-RPC 2.0 a mano (framing por línea, ciclo de vida initialize, luego notifications/initialized, después tools/list y finalmente tools/call).

- Comando de arranque:

```
python -m servers.pharmacy.local_stdio
```

- Catálogo de datos:

el servidor mantiene en memoria un catálogo de 4 medicamentos (paracetamol, ibuprofeno, loratadina, omeprazol), cada uno con descripción, síntomas asociados, precio y stock. El stock se actualiza en tiempo real conforme se realizan compras durante la sesión.

- Herramientas expuestas:

Herramienta	Parámetros	Descripción
-------------	------------	-------------

search_by_symptom	symptom (string, requerido)	Busca y recomienda medicamentos según un síntoma descrito en lenguaje natural.
get_medication_info	name (string, requerido)	Regresa descripción, precio y stock de un medicamento específico.
list_medications	(sin parámetros)	Lista el catálogo completo de medicamentos disponibles.
purchase_medication	name (string, requerido), quantity (integer ≥ 1, requerido)	Realiza la compra de una cantidad de un medicamento, validando stock disponible.

- Ejemplo de uso, flujo real capturado durante pruebas (conversación con contexto mantenido entre turnos):

Usuario: "tengo dolor de cabeza, ¿qué me recomiendas?"

```
// Solicitud MCP
{"jsonrpc":"2.0","id":3,"method":"tools/call","params":
  {"name":"search_by_symptom","arguments":{"symptom":"dolor de
cabeza"}}}
```

```
// Respuesta MCP
{"jsonrpc":"2.0","id":3,"result":{"content":[{"type":"text",
  "text":"Para 'dolor de cabeza' se recomienda: paracetamol,
ibuprofeno."}],
  "isError":false}}
```

Bot: "Para el dolor de cabeza se recomiendan: Paracetamol (Q15.00, 50 en stock) e Ibuprofeno (Q20.00, 40 en stock)..."

Usuario: "cómprame 2 de esos"

Bot: "¿Te refieres a 2 de Paracetamol, 2 de Ibuprofeno o 2 de cada uno?"

Usuario: "uno de cada uno"

```
// Solicitud MCP
{"jsonrpc":"2.0","id":6,"method":"tools/call","params":

{"name":"purchase_medication","arguments":{"name":"paracetamol","qua
ntity":1}}}
```

```
// Respuesta MCP
{"jsonrpc":"2.0","id":6,"result":{"content":[{"type":"text",
  "text":"Compra confirmada: 1 x paracetamol = Q15.00. Stock
restante: 49."}],
  "isError":false}}
```

Bot: "¡Compra realizada con éxito! 1 x Paracetamol: Q15.00 (stock restante 49), 1 x Ibuprofeno: Q20.00 (stock restante 39). Total: Q35.00"

Este ejemplo evidencia tanto el correcto funcionamiento del servidor MCP propio como el mantenimiento de contexto conversacional: en el segundo turno el usuario no repite los nombres de los medicamentos ('cómprame 2 de esos'), y el modelo resuelve la referencia correctamente contra el turno anterior, incluso pidiendo aclaración ante la ambigüedad en la cantidad.

2.4 Pharmacy MCP Server (propio): versión remota (Streamable HTTP)

Es el mismo servidor de farmacia de la sección 2.3, publicado en la nube para cumplir el inciso 6. La lógica de negocio (catálogo, definición de las herramientas y su ejecución) vive en `servers/pharmacy/logic.py` y la comparten, sin duplicar código, el transporte stdio (`local_stdio.py`) y el transporte HTTP (`remote_http.py`); lo único que cambia es cómo viajan los mensajes JSON-RPC. Las cuatro herramientas y sus parámetros son idénticos a los de la tabla de la sección 2.3 (la respuesta a `tools/list` capturada en la sección 4.3 lo confirma).

El servicio está publicado en Render, en <https://redes-proyecto1-9ubr.onrender.com>. En la entrega parcial se había previsto Google Cloud Run; el servicio finalmente se publicó en Render, un servicio de nube tipo PaaS. Implementa una versión simplificada del transporte Streamable HTTP de la especificación MCP 2025-11-25, sin ningún SDK de MCP: Flask se usa únicamente como framework HTTP genérico y todo el manejo de JSON-RPC y del ciclo de vida está escrito a mano.

- **Comando de arranque local (pruebas):** `python -m servers.pharmacy.remote_http` (escucha en el puerto de la variable `PORT`, 8080 por defecto).
- **Arranque en producción:** `gunicorn --bind 0.0.0.0:${PORT:-10000} --workers 1 --threads 4 servers.pharmacy.remote_http:app`. El repositorio incluye un `Dockerfile` (python:3.11-slim; dependencias en `servers/pharmacy/requirements.txt`: flask 3.1.0 y gunicorn 23.0.0). Se usa un solo worker porque el catálogo y las sesiones viven en la memoria del proceso.
- **Uso desde el chatbot:** basta definir `PHARMACY_REMOTE_URL=https://redes-proyecto1-9ubr.onrender.com` en el archivo `.env`. Con esa variable, `main.py` registra el servidor con `add_http_server()` en lugar de `add_server()`; el resto del chatbot lo usa exactamente igual que al servidor local.

Endpoints

Método	Ruta	Descripción	Códigos HTTP
POST	/mcp	Único endpoint del protocolo. Recibe un mensaje JSON-RPC 2.0 en el cuerpo (Content-Type: application/json).	200 respuesta JSON-RPC 202 notificación aceptada (sin cuerpo) 400 JSON inválido o sesión ausente/inválida
GET	/health	Comprobación de vida del servicio. Regresa <code>{"status":"ok","server":"pharmacy-mcp-server"}</code> .	200

Mensajes JSON-RPC soportados por POST /mcp

Método JSON-RPC	Tipo	Requiere Mcp-Session-Id	Respuesta
initialize	Solicitud (con id)	No (la crea)	200 + result: protocolVersion "2025-11-25", capabilities {"tools":{}}, serverInfo {"name":"pharmacy-mcp-server","version":"0.1.0"}. Además, la cabecera Mcp-Session-Id con un UUID v4.
notifications/initialized	Notificación (sin id)	Sí	202 Accepted, sin cuerpo.

Método JSON-RPC	Tipo	Requiere Mcp-Session-Id	Respuesta
tools/list	Solicitud	Sí	200 + result.tools: las 4 herramientas con su inputSchema.
tools/call	Solicitud	Sí	200 + result: {content:[{type:"text",text:...}], isError}.
Cualquier otro método con id	Solicitud	Sí	200 + error -32601 (método no soportado).
Cualquier otra notificación	Notificación	Sí	202 (se ignora).

Cabeceras y manejo de errores

- **Petición:** Content-Type: application/json siempre; Mcp-Session-Id: <uuid> en todas las peticiones excepto initialize. La sesión se guarda en memoria en el servidor.
- **Respuesta:** Content-Type: application/json; Mcp-Session-Id solo en la respuesta a initialize.

Código JSON-RPC	HTTP	Cuándo ocurre
-32700	400	El cuerpo de la petición no es JSON válido (Parse error).
-32000	400	Falta la cabecera Mcp-Session-Id o la sesión no existe.
-32601	200	Método no soportado o herramienta desconocida.
-32000	200	Excepción al ejecutar la herramienta ("Error ejecutando '...'").

Los errores de negocio (medicamento inexistente o stock insuficiente) no son errores de protocolo: se regresan como un resultado normal con isError: true y un texto explicativo, para que el LLM pueda leerlos y responderle al cliente.

Diferencias entre los dos transportes del mismo servidor

Aspecto	Local (stdio)	Remoto (Streamable HTTP)
Arranque del servidor	El chatbot lanza un subproceso (subprocess.Popen).	Ya está corriendo en la nube; el cliente solo envía peticiones.
Canal	stdin/stdout del subproceso.	HTTPS (TLS 1.3, puerto 443) hacia POST /mcp.
Delimitación de mensajes	Un JSON por línea (salto de línea).	Un JSON por cuerpo de petición HTTP (Content-Length).
Sesión	Implicita: dura lo que dura el proceso.	Cabecera Mcp-Session-Id, asignada en initialize.
Respuesta a una notificación	Ninguna.	HTTP 202 Accepted sin cuerpo JSON-RPC.
Clase cliente	MCPStdioClient	MCPHttpClient (requests.Session, keep-alive)

En ambos casos se conserva el mismo ciclo de vida (initialize, notifications/initialized, tools/list, tools/call) y la misma lógica de negocio.

Simplificaciones respecto a la especificación (documentadas por transparencia)

- Solo modo de respuesta única (un JSON por respuesta): no se implementa SSE ni el endpoint GET para streaming iniciado por el servidor. Las herramientas responden de inmediato, por lo que no hace falta.

- No se implementa el cierre explícito de sesión (DELETE); `MCPHttpClient.close()` no envía nada. Tampoco se valida la cabecera `MCP-Protocol-Version` ni se autentica al cliente: el servidor es una demostración académica sin datos sensibles.
- Sesiones y catálogo en memoria: si el servicio se reinicia, las sesiones previas dejan de ser válidas (el servidor responde 400) y el stock vuelve a sus valores iniciales.

Ejemplo real de intercambio (capturado con Wireshark, sección 4)

```
POST /mcp HTTP/1.1
Host: redes-proyecto1-9ubr.onrender.com
Content-Type: application/json
Mcp-Session-Id: c6a302c6-1564-4e1c-9bc9-427f3285d52c

{"jsonrpc": "2.0", "id": 3, "method": "tools/call",
 "params": {"name": "list_medications", "arguments": {}}}

HTTP/1.1 200 OK
Content-Type: application/json

{"id":3,"jsonrpc":"2.0","result":{"content":[{"text":"Catálogo disponible:
- paracetamol: Q15.00 (stock: 50)\n- ibuprofeno: Q20.00 (stock: 40)\n
- loratadina: Q18.00 (stock: 30)\n- omeprazol: Q25.00 (stock: 25)",
"type":"text"}],"isError":false}}
```

(Se muestran solo las cabeceras relevantes y los acentos ya decodificados; en la captura original el servidor los envía escapados como `\u00e1`.)

3. Interfaz de usuario web (extra)

Como extra del enunciado (sección 4.1) se implementó una interfaz web para el chatbot, además de la consola. Ambas son capas delgadas sobre el mismo `core/` (Chatbot, MCPManager y MCPLogger), por lo que la interfaz web no duplica lógica de negocio. El backend es una API HTTP en Flask (`interfaces/web.py`, arrancada con `main_web.py`) y el frontend es una aplicación React con Vite (`frontend/`).

3.1 Cómo ejecutarla

```
# Terminal 1 (raíz del proyecto): backend
pip install flask
python -m main_web          # API en http://localhost:5050

# Terminal 2: interfaz
cd frontend
npm install
npm run dev                 # abrir http://localhost:5173
```

Alternativa para la demostración: con `npm run build` dentro de `frontend/`, basta `python -m main_web` y abrir `http://localhost:5050` (Flask sirve el build, sin Vite). Para usar el servidor remoto de farmacia se agrega `PHARMACY_REMOTE_URL` al `.env`, como en la sección 2.4. El puerto del backend se cambia con la variable `WEB_PORT` (y el proxy de `frontend/vite.config.js`).

3.2 API del backend

Método	Ruta	Descripción
GET	/api/status	Servidores MCP conectados, si son locales o remotos, y sus herramientas.
GET	/api/catalog	Catálogo de la farmacia. Llama directamente a la herramienta MCP <code>list_medications</code> , sin pasar por el LLM (no consume cuota de Gemini); la llamada también queda en el log MCP.
GET	/api/history	Mensajes de la conversación actual, para que el historial sobreviva a recargar la página.
POST	/api/chat	Cuerpo <code>{"message": "..."}.</code> Regresa <code>{"reply": "...", "tool_calls": [{server, name, arguments}]}</code> .
POST	/api/reset	Borra el contexto de la conversación (equivale a <code>/reset</code> en la consola).
GET	/api/log?since=N	Entradas nuevas del log MCP desde el índice N; la interfaz lo consulta cada 2 segundos.

3.3 Funcionalidades

- **Chat con contexto de sesión**, respuestas con formato Markdown y “chips” que muestran qué herramienta MCP y de qué servidor usó el LLM en cada respuesta.
- **Catálogo de la farmacia** con precio, unidades y barra de stock; avisa “Poco stock” con 10 unidades o menos y se actualiza solo después de una compra.
- **Servidores MCP** (filesystem, git, pharmacy), cada uno con su etiqueta “local · stdio” o “remoto · HTTP” y la lista de sus herramientas.
- **Log MCP** (requisito 3): panel lateral con las solicitudes, respuestas, notificaciones y errores en vivo, filtro por servidor y el JSON-RPC completo al expandir cada entrada. El botón muestra el número de entradas.
- Indicador de estado de conexión con el backend, botón de nueva conversación, diseño adaptable a móvil e historial que se conserva al recargar (el contexto vive en el servidor).

3.4 Decisiones de diseño (HCI)

- **Psicología del color:** celeste (confianza y calma) y verde menta (salud y bienestar) sobre fondos muy claros, para transmitir una farmacia limpia y confiable. Texto oscuro azul-grisáceo sobre fondo claro para mantener buen contraste; los botones de acción usan tonos más oscuros del mismo color para que el texto blanco sea legible, y los pasteles se reservan para fondos.
- **Organización de los elementos:** tres zonas con jerarquía clara. A la izquierda, el contexto (catálogo y servidores); al centro, la conversación como foco principal; y a la derecha, el log MCP, que es información técnica y por eso permanece oculto hasta que se pide (botón “Log MCP”). El encabezado concentra el estado y las acciones globales.
- **Usabilidad y retroalimentación:** los chips hacen visible lo que hizo el sistema (transparencia); el catálogo refleja de inmediato el efecto de una compra; hay indicador de escritura, mensajes de error claros y Enter para enviar / Shift+Enter para nueva línea.
- **Accesibilidad:** foco visible en los controles, áreas táctiles de al menos 40 px, región aria-live en el chat, etiquetas aria-label y tipografía Nunito con respaldo a Segoe UI cuando no hay Internet.
- **Uso responsable:** la interfaz muestra de forma permanente el aviso “Demostración académica: las recomendaciones no sustituyen la consulta con un profesional de la salud”.

3.5 Capturas de la interfaz

Las Figuras 1 y 2 muestran una misma conversación. En la primera, ante “Hola, me duele la cabeza”, el modelo encadenó tres herramientas del servidor de farmacia (`search_by_symptom` y dos `get_medication_info`, visibles como chips) y presentó Paracetamol e Ibuprofeno con su descripción, precio y stock. En la segunda se comprueba el contexto de sesión: ante “me puedes dar 4?” el asistente entiende que se refiere a los medicamentos recomendados y pide aclarar cuál; con “me gustaría 4 de cada uno” ejecuta dos veces `purchase_medication`, resume el pedido (Q140.00) y el catálogo pasa de 50 a 46 unidades de Paracetamol y de 40 a 36 de Ibuprofeno.



Figura 1. Consulta por síntoma: chips de las herramientas usadas y catálogo con el stock inicial. Contador del Log MCP: 27.

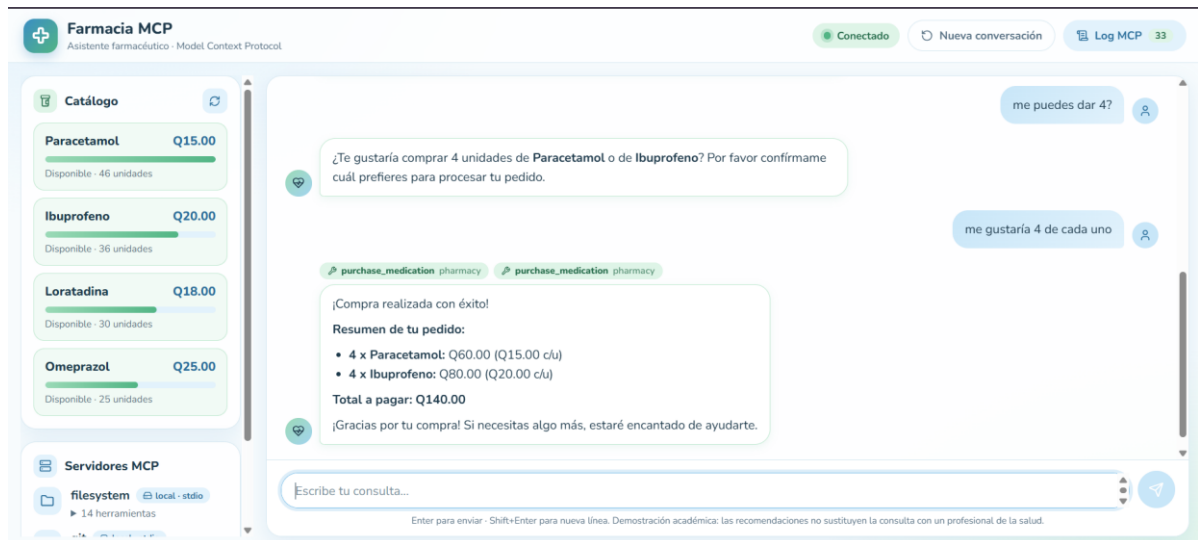


Figura 2. Compra con contexto de sesión: dos llamadas a `purchase_medication` y catálogo actualizado. El contador del Log MCP pasó de 27 a 33 entradas: 4 de las dos compras (solicitud y respuesta de cada una) y 2 de la actualización automática del catálogo.

4. Análisis de la comunicación con Wireshark (incisos 7 y 9)

4.1 Escenario y metodología de captura

La captura se realizó en Windows con Wireshark, sobre la interfaz Wi-Fi Intel(R) Wi-Fi 6E AX211 160MHz, mientras el chatbot (con `PHARMACY_REMOTE_URL` apuntando al servidor de Render) arrancaba y consultaba el catálogo de la farmacia. Las horas son hora local de Guatemala (UTC-6); coinciden, con diferencia de un segundo, con la cabecera Date (GMT) de las respuestas del servidor.

- **Captura A** (filtro `tcp.stream eq 58`): una conexión TCP 192.168.0.11:61135 ↔ 216.24.57.18:443, del 13:39:12 al 13:40:52 del 22 de septiembre de 2026. Contiene el handshake TCP y TLS y los cuatro mensajes MCP: `initialize`, `notifications/initialized`, `tools/list` y una llamada `tools/call` a `list_medications`, la misma que la interfaz web usa para llenar el catálogo.
- **Captura B**: otra ejecución del chatbot (conexión 56612 ↔ 443, del 13:48:19 al 13:48:51), usada para comprobar que el intercambio se repite igual y para observar el cierre de la conexión.

4.2 Justificación: por qué el HTTP no se lee directamente en la red y cómo se descifró

El cliente consume el servidor remoto por HTTPS (puerto 443): todo el diálogo HTTP viaja dentro de TLS 1.3, como se aprecia en las Figuras 4 y 8 (Client Hello con SNI, Server Hello y registros “TLS segment of a reassembled PDU”). Por diseño, TLS cifra la capa de aplicación, así que quien solo observa la red no puede leer las cabeceras HTTP ni los mensajes JSON-RPC. Sin las claves de sesión, Wireshark mostraría únicamente registros TLS de tipo Application Data, y de la comunicación MCP solo serían observables las direcciones IP, los puertos, los tamaños, los tiempos y el nombre del servidor (SNI), que el cliente envía sin cifrar en el Client Hello.

Por esta razón, el contenido HTTP y JSON-RPC no se puede leer a simple vista en la captura de red. Tampoco se obtuvo interceptando el tráfico (proxy o ataque de intermediario), sino de forma pasiva, con el mecanismo estándar de exportar desde el propio cliente las claves de sesión TLS. Así se cumple el inciso 7 sin exponer el servicio por HTTP plano ni debilitar su seguridad. El procedimiento fue:

- En `core/mcp_client.py`, el cliente HTTP (`MCPHttpClient`) monta un adaptador (`_KeyLogAdapter`) que, si existe la variable de entorno `SSLKEYLOGFILE`, configura el contexto TLS para escribir allí las claves de sesión.
- Se ejecutó el chatbot con `SSLKEYLOGFILE` definida mientras Wireshark capturaba.
- En Wireshark se indicó ese archivo en Edición → Preferencias → Protocols → TLS → (Pre)-Master-Secret log filename. Con las claves, Wireshark descifra los registros TLS y aplica los disectores HTTP y JSON.

Evidencias de que el descifrado funcionó: (1) los frames de datos aparecen como HTTP/JSON y se puede usar Seguir → Secuencia HTTP (Figuras 5 a 7); (2) los mensajes del handshake TLS 1.3 posteriores al Server Hello (Encrypted Extensions, Certificate, Certificate Verify, Finished), que en TLS 1.3 ya viajan cifrados, aparecen disecados en el frame 1327; (3) el panel de bytes ofrece las pestañas “Decrypted TLS” y “Reassembled TLS” (Figura 4).

Limitaciones que conviene declarar:

- Alcance: solo se pueden descifrar las conexiones creadas por el proceso que exportó las claves (aquí, el cliente del servidor MCP remoto). Otras conexiones presentes en la captura no forman parte de este análisis.
- Lo que muestra Wireshark es una representación: las respuestas llegan con Content-Encoding: `gzip` y Transfer-Encoding: `chunked`, y Wireshark las descomprime y reensambla para mostrarlas; en el cable viajan cifradas y comprimidas. Por eso los

tamaños de frame de la lista no coinciden con el tamaño del JSON (se explican en la sección 4.4.3).

- Seguridad: el archivo de claves permite a quien tenga también la captura leer todo el tráfico, así que es material sensible y conviene borrarlo al terminar. El .gitignore del proyecto ya excluye *.pcap, *.pcapng y *.log.

4.3 Clasificación de los mensajes JSON-RPC (inciso 7)

En la captura A se observan cuatro intercambios HTTP dentro de una misma conexión TCP/TLS (keep-alive). En MCP sobre HTTP, cada mensaje JSON-RPC del cliente viaja en un POST /mcp y cada respuesta, en el cuerpo de la respuesta HTTP. La Figura 3 resume la secuencia y la Tabla siguiente clasifica cada mensaje.

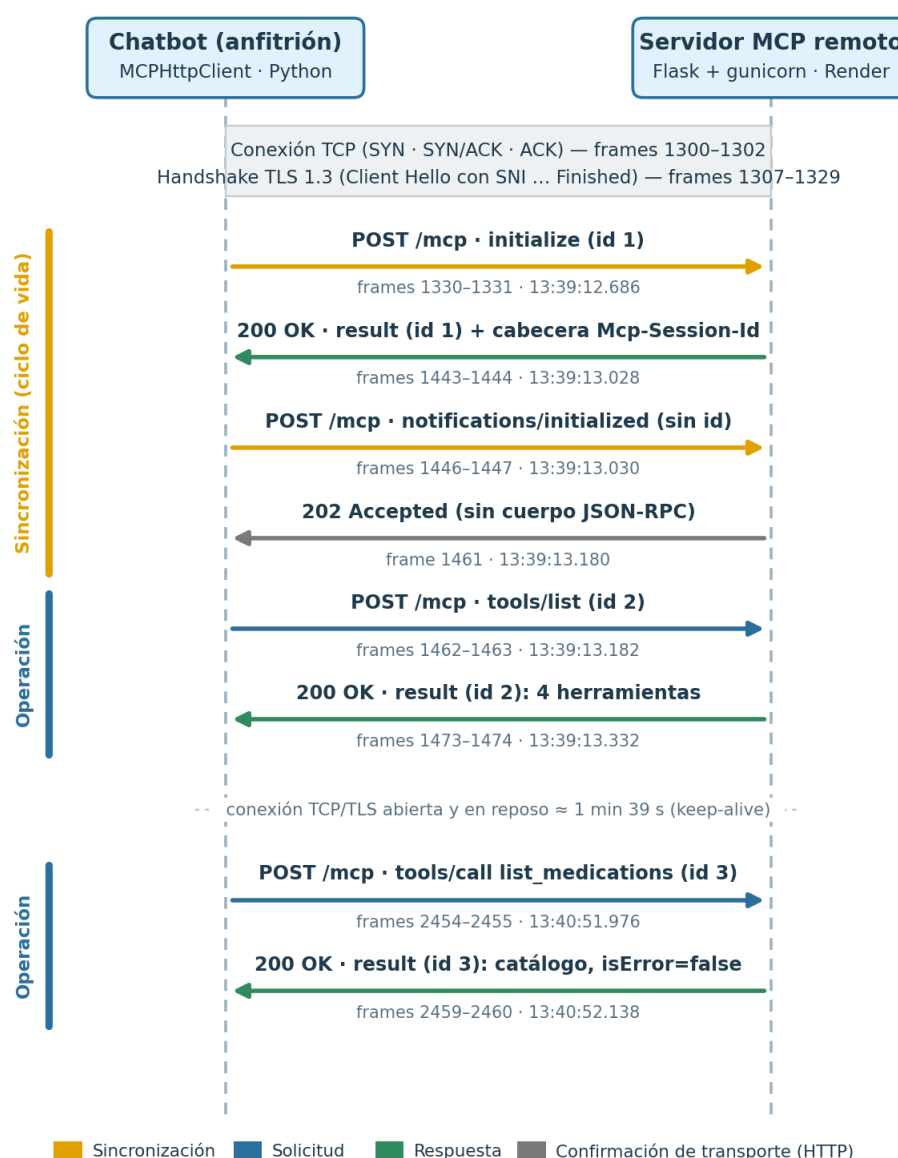


Figura 3. Secuencia de mensajes entre el anfitrión y el servidor remoto (captura A, tcp.stream eq 58).

Frames	Hora local	Sentido	Mensaje JSON-RPC	Clasificación
1330–1331	13:39:12.686	Cliente → Servidor	initialize (id 1): protocolVersion 2025-11-25, clientInfo mi-chatbot 0.1.0	Sincronización (negociación de versión y capacidades)

Frames	Hora local	Sentido	Mensaje JSON-RPC	Clasificación
1443–1444	13:39:13.028	Servidor → Cliente	Respuesta al id 1: capabilities {tools}, serverInfo pharmacy-mcp-server 0.1.0, cabecera Mcp-Session-Id (HTTP 200)	Respuesta (cierra la negociación de initialize)
1446–1447	13:39:13.030	Cliente → Servidor	notifications/initialized (sin id)	Sincronización (notificación: el cliente declara que ya está listo)
1461	13:39:13.180	Servidor → Cliente	Sin mensaje JSON-RPC: HTTP 202 Accepted	Confirmación de transporte
1462–1463	13:39:13.182	Cliente → Servidor	tools/list (id 2)	Solicitud
1473–1474	13:39:13.332	Servidor → Cliente	Respuesta al id 2: las 4 herramientas con su inputSchema (HTTP 200)	Respuesta
2454–2455	13:40:51.976	Cliente → Servidor	tools/call (id 3): list_medications, arguments {}	Solicitud
2459–2460	13:40:52.138	Servidor → Cliente	Respuesta al id 3: catálogo en content[0].text, isError false (HTTP 200)	Respuesta

Criterio de clasificación: los mensajes de **sincronización** son los del ciclo de vida que acuerdan la versión del protocolo y las capacidades antes de operar (initialize y notifications/initialized); las **solicitudes** operativas llevan id y method (tools/list, tools/call); las **respuestas** repiten el id de su solicitud y traen result (o error); y una **notificación** no lleva id ni tiene respuesta JSON-RPC, por lo que el servidor solo la confirma con HTTP 202 a nivel de transporte. Esta clasificación coincide con lo registrado por el log MCP del propio chatbot.

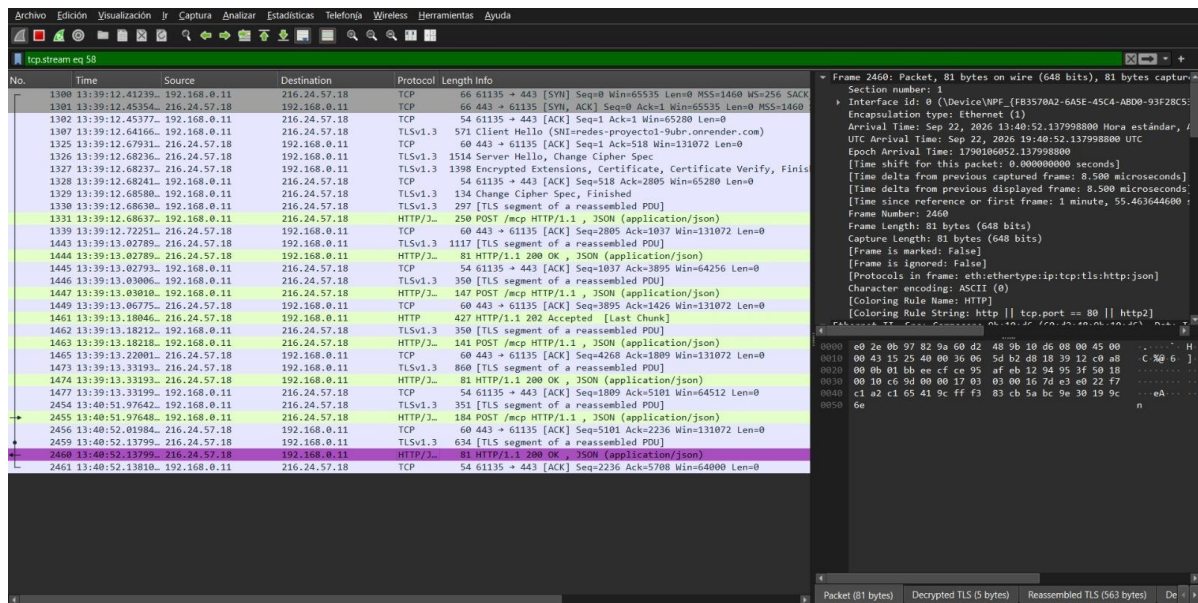


Figura 4. Captura A (tcp.stream eq 58): handshake TCP (1300–1302), handshake TLS 1.3 (1307–1329) y los cuatro intercambios MCP. El frame 2460 (resaltado) muestra a la derecha su detalle y su volcado hexadecimal.

Contenido de los mensajes, tal como los muestra Wireshark con el descifrado activo (Figuras 5 a 7). Los mensajes del cliente llevan espacios tras : y ; los del servidor son compactos, con las llaves en orden alfabético:

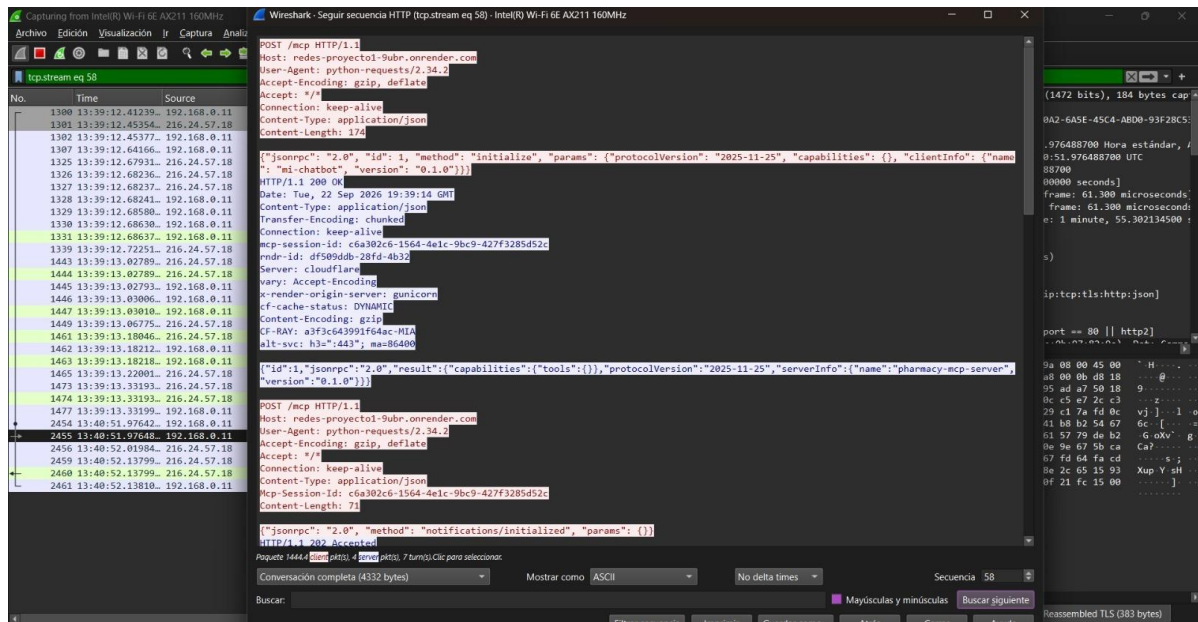


Figura 5. Seguir → Secuencia HTTP (parte 1): inicialize con su respuesta (y la cabecera mcp-session-id) y notifications/initialized.

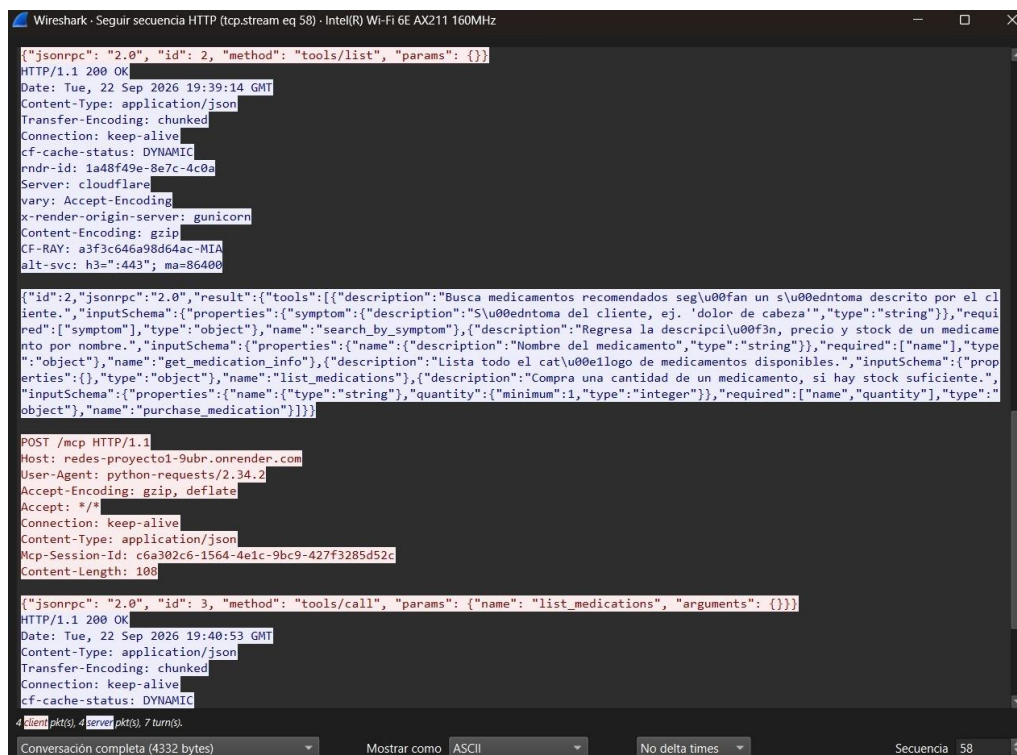


Figura 6. Seguir → Secuencia HTTP (parte 2): tools/list con su respuesta y el inicio de tools/call.

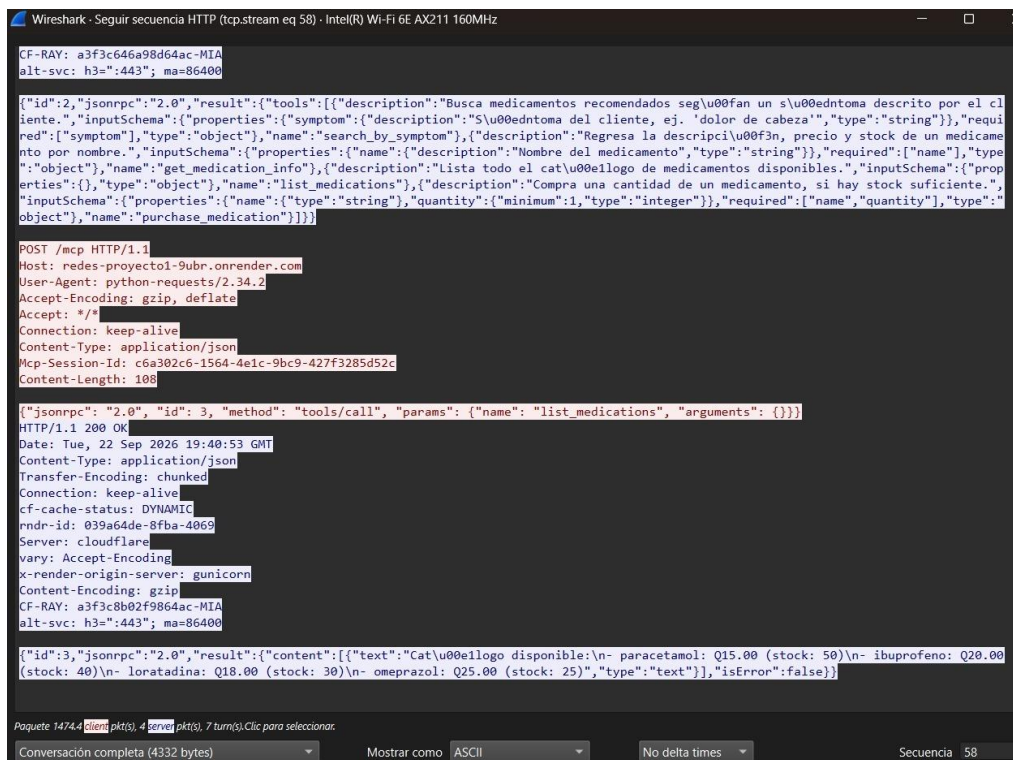


Figura 7. Seguir → Secuencia HTTP (parte 3): la respuesta a tools/call (list_medications). Al pie, 4 paquetes del cliente y 4 del servidor en la conversación completa (4332 bytes).

```

→ {"jsonrpc": "2.0", "id": 1, "method": "initialize", "params": {"protocolVersion": "2025-11-25",
  "capabilities": {}, "clientInfo": {"name": "mi-chatbot", "version": "0.1.0"}}}
← {"id":1,"jsonrpc":"2.0","result":{"capabilities":{"tools":{}},"protocolVersion":"2025-11-25",
  "serverInfo":{"name":"pharmacy-mcp-server","version":"0.1.0"}}}      (+ Mcp-Session-Id)
→ {"jsonrpc": "2.0", "method": "notifications/initialized", "params": {}} ← HTTP 202 Accepted
→ {"jsonrpc": "2.0", "id": 2, "method": "tools/list", "params": {}}
←
{"id":2,"jsonrpc":"2.0","result":{"tools":[{"name":"search_by_symptom"},{... "name":"get_medication_info"},
  {... "name":"list_medications"},{... "name":"purchase_medication"}]}}
→ {"jsonrpc": "2.0", "id": 3, "method": "tools/call", "params": {"name": "list_medications", "arguments":
  {}}}
← {"id":3,"jsonrpc":"2.0","result":{"content":[{"text":"Catálogo disponible:
  ...", "type":"text"}], "isError":false}}

```

Repetibilidad (captura B). En otra ejecución se observa la misma secuencia con los mismos tamaños de frame (POST de 250, 147 y 141 bytes; respuestas de 81 y 427 bytes) y, además,

el cierre de la conexión con FIN (frames 1310–1312). En esa ejecución no hubo tools/call, por lo que la conexión termina tras tools/list.

No.	Time	Source	Destination	Protocol	Length	Info
366	13:48:19.75751...	192.168.0.11	216.24.57.18	TCP	66	56612 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK
370	13:48:19.80300...	216.24.57.18	192.168.0.11	TCP	66	443 → 56612 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460
371	13:48:19.80310...	192.168.0.11	216.24.57.18	TCP	54	56612 → 443 [ACK] Seq=1 Ack=1 Win=65280 Len=0
383	13:48:20.01937...	192.168.0.11	216.24.57.18	TLSv1.3	571	Client Hello (SNI=redes-proyecto1-9ubr.onrender.com)
384	13:48:20.06684...	216.24.57.18	192.168.0.11	TCP	60	443 → 56612 [ACK] Seq=1 Ack=518 Win=131072 Len=0
385	13:48:20.06879...	216.24.57.18	192.168.0.11	TLSv1.3	1514	Server Hello, Change Cipher Spec
386	13:48:20.06879...	216.24.57.18	192.168.0.11	TLSv1.3	1397	Encrypted Extensions, Certificate, Certificate Verify, Finis
387	13:48:20.06882...	192.168.0.11	216.24.57.18	TCP	54	56612 → 443 [ACK] Seq=518 Ack=2804 Win=65280 Len=0
388	13:48:20.07443...	192.168.0.11	216.24.57.18	TLSv1.3	134	Change Cipher Spec, Finished
389	13:48:20.07507...	192.168.0.11	216.24.57.18	TLSv1.3	297	[TLS segment of a reassembled PDU]
390	13:48:20.07518...	192.168.0.11	216.24.57.18	HTTP/1.1	250	POST /mcp HTTP/1.1 , JSON (application/json)
394	13:48:20.12359...	216.24.57.18	192.168.0.11	TCP	60	443 → 56612 [ACK] Seq=2804 Ack=1037 Win=131072 Len=0
437	13:48:20.23580...	216.24.57.18	192.168.0.11	TLSv1.3	1117	[TLS segment of a reassembled PDU]
438	13:48:20.23580...	216.24.57.18	192.168.0.11	HTTP/1.1	81	HTTP/1.1 200 OK , JSON (application/json)
439	13:48:20.23585...	192.168.0.11	216.24.57.18	TCP	54	56612 → 443 [ACK] Seq=1037 Ack=3894 Win=64256 Len=0
440	13:48:20.23966...	192.168.0.11	216.24.57.18	TLSv1.3	350	[TLS segment of a reassembled PDU]
441	13:48:20.23977...	192.168.0.11	216.24.57.18	HTTP/1.1	147	POST /mcp HTTP/1.1 , JSON (application/json)
447	13:48:20.28642...	216.24.57.18	192.168.0.11	TCP	60	443 → 56612 [ACK] Seq=3894 Ack=1426 Win=131072 Len=0
463	13:48:20.58277...	216.24.57.18	192.168.0.11	HTTP	427	HTTP/1.1 202 Accepted [Last Chunk]
464	13:48:20.58664...	192.168.0.11	216.24.57.18	TLSv1.3	350	[TLS segment of a reassembled PDU]
465	13:48:20.58669...	192.168.0.11	216.24.57.18	HTTP/1.1	141	POST /mcp HTTP/1.1 , JSON (application/json)
466	13:48:20.63031...	216.24.57.18	192.168.0.11	TCP	60	443 → 56612 [ACK] Seq=4267 Ack=1809 Win=131072 Len=0
467	13:48:20.74154...	216.24.57.18	192.168.0.11	TLSv1.3	860	[TLS segment of a reassembled PDU]
468	13:48:20.74154...	216.24.57.18	192.168.0.11	HTTP/1.1	81	HTTP/1.1 200 OK , JSON (application/json)
469	13:48:20.74159...	192.168.0.11	216.24.57.18	TCP	54	56612 → 443 [ACK] Seq=1809 Ack=5100 Win=64512 Len=0
1310	13:48:51.73440...	192.168.0.11	216.24.57.18	TCP	54	56612 → 443 [FIN, ACK] Seq=1809 Ack=5100 Win=64512 Len=0
1311	13:48:51.79690...	216.24.57.18	192.168.0.11	TCP	60	443 → 56612 [FIN, ACK] Seq=5100 Ack=1810 Win=131072 Len=0
1312	13:48:51.79695...	192.168.0.11	216.24.57.18	TCP	54	56612 → 443 [ACK] Seq=1810 Ack=5101 Win=64512 Len=0

Figura 8. Captura B: establecimiento de la conexión, handshake TLS 1.3, los mensajes de sincronización y de solicitud, y el cierre con FIN/ACK.

4.4 Análisis por capas (inciso 9)

La Figura 9 descompone el frame 2455 (la parte del cuerpo del POST /mcp de tools/call) en las capas por las que pasa un mensaje MCP en el cable. Las secciones siguientes explican qué ocurre en cada una, con evidencia de la captura.

Frame 2455 (cliente → servidor): parte del POST /mcp con tools/call — 184 B en el cable

Los campos pequeños (TLS 5 B, tipo 1 B) no están a escala.

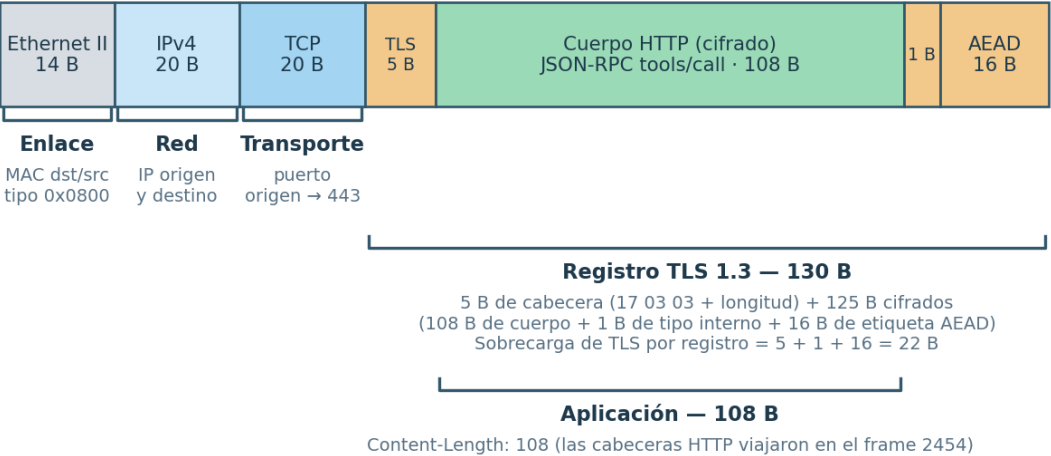


Figura 9. Encapsulamiento del frame 2455: Ethernet II, IPv4, TCP y un registro TLS 1.3 que contiene el cuerpo JSON-RPC.

4.4.1 Capa de enlace (Ethernet II)

Wireshark reporta “Encapsulation type: Ethernet (1)” para la interfaz Wi-Fi. Aunque el medio físico es inalámbrico, en modo de captura normal Wireshark presenta las tramas con un encabezado Ethernet II de 14 bytes: MAC destino (6 B), MAC origen (6 B) y EtherType (2 B). En el frame 2460 (81 bytes, servidor → cliente), el volcado hexadecimal (Figura 4) muestra destino e0:2e:0b:97:82:9a (la tarjeta de red del equipo), origen 60:d2:48:9b:10:d6 y EtherType 0x0800 (IPv4).

El origen no es la MAC del servidor de Render, sino la del dispositivo del salto anterior en la red local (presumiblemente el router). Las direcciones MAC solo tienen significado dentro del enlace local: cada salto reescribe el encabezado de enlace, mientras que la dirección IP se conserva de extremo a extremo. La función de esta capa es entregar la trama al siguiente salto, delimitarla y direccionarla a nivel físico; no sabe nada de TCP, TLS ni MCP.

4.4.2 Capa de red (IPv4)

Cada segmento TCP viaja en un datagrama IPv4. En el frame 2460: versión 4 e IHL de 5 (encabezado de 20 bytes), longitud total de 67 bytes, indicador DF (no fragmentar), TTL de 54, protocolo 6 (TCP) y checksum de cabecera 0x5db2. El origen es 216.24.57.18 (servidor) y el destino 192.168.0.11 (el equipo).

- **Direccionamiento y NAT:** 192.168.0.11 es una dirección privada (RFC 1918) de la red local; el router la traduce (NAT) a su dirección pública para salir a Internet. 216.24.57.18 es el punto de entrada público del servicio: las cabeceras `Server: cloudflare`, `CF-RAY` y `x-render-origin-server: gunicorn` indican que hay un proxy (Cloudflare) delante del servidor de aplicación (gunicorn).
- **Enrutamiento:** los routers deciden el siguiente salto con la IP destino y cada uno decrementa el TTL. El paquete llegó con 54; si el valor inicial fue 64 (habitual), cruzó unos 10 saltos (estimación).
- **Sin fragmentación:** el MSS de 1460 bytes negociado en el handshake TCP mantiene los datagramas dentro de los 1500 bytes de MTU (el frame más grande observado mide 1514 bytes = 14 + 1500) y el bit DF está activo.
- **Nombres:** el cliente usa el nombre `redes-proyecto1-9ubr.onrender.com`. La resolución DNS ocurre antes de abrir la conexión (no aparece en el filtro `tcp.stream eq 58`) y el nombre reaparece como SNI (TLS) y como cabecera `Host` (HTTP).

4.4.3 Capa de transporte (TCP y TLS 1.3)

TCP. Ofrece una conexión confiable y ordenada entre dos puertos: el destino es el 443 (HTTPS) y el origen es un puerto efímero (61135 en la captura A y 56612 en la B).

- **Establecimiento de tres vías** (frames 1300–1302): SYN del cliente (66 bytes, con las opciones MSS = 1460, escala de ventana = 256 y SACK permitido), SYN/ACK del servidor y ACK. El intervalo SYN → SYN/ACK (13:39:12.41239 → .45354) da un RTT de red de aproximadamente 41 ms.
- **Confiabilidad:** cada byte se numera y el receptor confirma con ACK acumulativos. La tabla siguiente reconstruye la conversación: en todos los tramos, el número de ACK (o de Seq) coincide con 1 más la suma de bytes de datos enviados en esa dirección, y la lista no muestra retransmisiones.
- **ACK antes que la respuesta de aplicación:** el servidor confirma la recepción de cada POST en unos 36–43 ms (frames 1339, 1449, 1465 y 2456, aproximadamente un RTT), mientras que la respuesta HTTP tarda unos 150–160 ms (unos 342 ms en `initialize`, la primera petición). TCP garantiza la entrega con independencia de cuánto tarde la aplicación en procesar.
- **Una sola conexión para toda la sesión:** los cuatro mensajes usan la misma conexión (`Connection: keep-alive`, `requests.Session`), incluso con 1 min 39 s de inactividad entre `tools/list` y `tools/call`. El costo del handshake TCP y TLS se paga una sola vez.
- **Segmentación:** cada petición HTTP sale en dos segmentos (primero las cabeceras y luego el cuerpo), y cada respuesta también en dos (cabeceras con el cuerpo comprimido, y un último registro de 5 bytes). Wireshark marca los primeros como “TLS segment of a reassembled PDU” y muestra el HTTP/JSON en el último.

- **Cierre:** en la captura B, el cliente inicia el cierre con FIN,ACK (frame 1310, 13:48:51.73), el servidor responde FIN,ACK (1311) y el cliente confirma con ACK (1312), unos 31 s después de la última respuesta. No proviene de un mensaje MCP (MCPHttpClient.close() no termina la sesión), sino del cierre del socket por parte del cliente, presumiblemente al finalizar el proceso del chatbot.

Cuenta de bytes de la captura A (datos TCP = tamaño del frame – 54 bytes de encabezados Ethernet, IP y TCP; números relativos de Wireshark):

Frames	Sentido	Frame (B)	Datos TCP (B)	Ack/Seq resultante	Contenido
1307	C → S	571	517	Ack 518 (frame 1325)	Client Hello
1326 + 1327	S → C	1514 + 1398	1460 + 1344 = 2804	Ack 2805 (frame 1328)	Server Hello ... Finished del servidor
1329 – 1331	C → S	134 + 297 + 250	80 + 243 + 196 = 519	Ack 1037 (frame 1339)	Finished del cliente + POST initialize
1443 + 1444	S → C	1117 + 81	1063 + 27 = 1090	Ack 3895 (frame 1445)	200 OK de initialize
1446 + 1447	C → S	350 + 147	296 + 93 = 389	Ack 1426 (frame 1449)	POST notifications/initialized
1461	S → C	427	373	Seq 4268 (frame 1465)	202 Accepted
1462 + 1463	C → S	350 + 141	296 + 87 = 383	Ack 1809 (frame 1465)	POST tools/list
1473 + 1474	S → C	860 + 81	806 + 27 = 833	Ack 5101 (frame 1477)	200 OK de tools/list
2454 + 2455	C → S	351 + 184	297 + 130 = 427	Ack 2236 (frame 2456)	POST tools/call
2459 + 2460	S → C	634 + 81	580 + 27 = 607	Ack 5708 (frame 2461)	200 OK de tools/call

TLS 1.3. Corre sobre TCP y agrega confidencialidad, integridad y autenticación del servidor.

- **Handshake en un viaje de ida y vuelta (1-RTT):** Client Hello (571 B, con el SNI redes-proyecto1-9ubr.onrender.com en claro); Server Hello + Change Cipher Spec (1514 B); Encrypted Extensions + Certificate + Certificate Verify + Finished (1398 B; el certificado ya viaja cifrado en TLS 1.3); y Change Cipher Spec + Finished del cliente (134 B). El Change Cipher Spec se envía solo por compatibilidad. El primer POST sale unos 0.6 ms después del Finished del cliente (frames 1329 → 1331).
- **Registros de datos:** cada uno lleva un encabezado de 5 bytes (tipo 0x17 Application Data, versión 0x0303 heredada por compatibilidad y longitud), los datos cifrados, 1 byte de tipo interno y 16 bytes de etiqueta de autenticación (AEAD). La sobrecarga es de 22 bytes por registro y se verifica en la captura: Content-Length 174, 71 y 108 (Figuras 5 y 7) corresponden a registros de 196, 93 y 130 bytes, y el cuerpo de tools/list (65 bytes) a uno de 87 bytes.
- **Ejemplo del frame 2460 (Figura 4):** el payload TCP empieza con 17 03 03 00 16 seguido de 22 bytes cifrados; Wireshark lo descifra a 5 bytes (22 – 1 – 16), consistente con el terminador de una respuesta chunked (0\r\n\r\n).

4.4.4 Capa de aplicación (HTTP/1.1 y JSON-RPC 2.0 / MCP)

- **Petición HTTP:** POST /mcp HTTP/1.1 CON Host: redes-proyecto1-9ubr.onrender.com, User-Agent: python-requests/2.34.2, Accept: */*, Accept-Encoding: gzip, deflate, Connection: keep-alive, Content-Type: application/json y Content-Length (174, 71 y 108 bytes en los mensajes de las Figuras 5 y 7).
- **Sesión sobre un protocolo sin estado:** HTTP no guarda estado, así que MCP usa la cabecera Mcp-Session-Id. El servidor genera un UUID en la respuesta a initialize (c6a302c6-1564-4e1c-9bc9-427f3285d52c en la captura) y el cliente lo reenvía en todas las peticiones siguientes; sin él, el servidor responde 400 (sección 2.4).
- **Respuesta HTTP:** 200 OK con Content-Type: application/json, Transfer-Encoding: chunked y Content-Encoding: gzip (el cliente anuncia Accept-Encoding: gzip, deflate); la notificación recibe 202 Accepted sin mensaje JSON-RPC. Las cabeceras

Server: cloudflare, CF-RAY (con sufijo -MIA), cf-cache-status: DYNAMIC (no se cachea), rndr-id, x-render-origin-server: gunicorn y alt-svc: h3=":443" (anuncia HTTP/3, que este cliente no usa) revelan la infraestructura intermedia.

- **JSON-RPC 2.0 / MCP:** las solicitudes llevan id y method; las respuestas repiten el id y traen result; la notificación no lleva id. El servidor serializa de forma compacta, con llaves ordenadas y caracteres no ASCII escapados (por ejemplo \u00fa), lo cual es equivalente para cualquier parser JSON.
- **Frente a stdio:** los mismos mensajes viajan sin TLS, TCP ni HTTP (una línea por mensaje, sin Mcp-Session-Id ni 202). El transporte HTTP agrega sesión, cifrado y latencia de red (unos 150 ms por mensaje en esta captura).

4.5 Resumen por capas

Capa	Protocolo	Qué se observó	Evidencia
Enlace	Ethernet II (sobre Wi-Fi)	MAC origen 60:d2:48:9b:10:d6 (salto anterior) y destino e0:2e:0b:97:82:9a (equipo); EtherType 0x0800; trama de 81 B.	Frame 2460, Figura 4
Red	IPv4	192.168.0.11 ↔ 216.24.57.18; TTL 54; DF; protocolo 6; sin fragmentación (MSS 1460).	Frame 2460, Figura 4
Transporte	TCP + TLS 1.3	Puertos 61135 → 443; tres vías; ACK acumulativos exactos; una sola conexión reutilizada; handshake TLS 1-RTT con SNI; 22 B de sobrecarga por registro; cierre con FIN.	Frames 1300–1331, Figuras 4 y 8, tabla de bytes
Aplicación	HTTP/1.1 + JSON-RPC 2.0 (MCP 2025-11-25)	POST /mcp con Mcp-Session-Id; respuestas 200 y 202; mensajes initialize, notifications/initialized, tools/list y tools/call.	Figuras 3 y 5 a 7

5. Conclusiones y comentarios (inciso 10)

- **Logros.** Se completaron los siete requerimientos funcionales del enunciado y el extra de interfaz de usuario: un chatbot conectado a la API de Google Gemini, con contexto de sesión y log de todos los mensajes MCP; un cliente MCP propio, sin SDKs, con dos transportes (stdio y Streamable HTTP simplificado) y el ciclo de vida completo (initialize, notifications/initialized, tools/list, tools/call); la integración de los servidores oficiales Filesystem y Git; un servidor propio de farmacia con cuatro herramientas, disponible en local y en remoto con la misma lógica de negocio; el análisis del tráfico con Wireshark; y una interfaz web.
- **Sobre el protocolo.** MCP separa lo que se dice (mensajes JSON-RPC) de cómo viaja (el transporte). El servidor de farmacia responde igual por stdio que por HTTPS, y para llevarlo a la nube solo hizo falta un adaptador de transporte (remote_http.py) sobre la misma lógica. Lo que sí cambia con el transporte remoto es la gestión de sesión (cabecera Mcp-Session-Id), la respuesta 202 a las notificaciones y el costo de red: en la captura, cada mensaje tardó unos 150–160 ms, y unos 340 ms el primero.
- **Sobre Wireshark y la seguridad.** El tráfico MCP remoto no es legible en la red porque viaja en TLS 1.3; para verlo fue necesario exportar las claves de sesión desde el cliente (sección 4.2). El análisis permitió comprobar con bytes reales lo que se estudia en teoría: la cuenta exacta de bytes TCP, los 22 bytes de sobrecarga por registro TLS, la reutilización de una sola conexión y el cierre con FIN. También es una buena razón para exponer servidores MCP remotos únicamente por HTTPS.
- **Dificultades.** (1) En Windows, subprocess.Popen no resuelve automáticamente ejecutables .cmd como npx, por lo que fue necesario resolver la ruta con shutil.which antes de lanzar el proceso. (2) El límite de iteraciones del ciclo de function calling se tuvo que ajustar, porque el modelo no siempre agrupa varias llamadas en un mismo turno y las pide de forma secuencial. (3) El cambio de proveedor de LLM a mitad del

desarrollo (de Anthropic a Google Gemini) obligó a adaptar el formato de mensajes y de function calling; la arquitectura en capas limitó el cambio a `llm_client.py` y `chatbot.py`. (4) El transporte HTTP exigió manejar la sesión en una cabecera de la respuesta a `initialize` (no en el cuerpo) y responder 202 a las notificaciones, reutilizando la lógica sin duplicarla (`logic.py`). (5) El HTTP cifrado no era legible en Wireshark, lo que se resolvió con `SSLKEYLOGFILE`. (6) El despliegue previsto en Cloud Run terminó en Render, y como el estado vive en memoria el servicio corre con un solo worker de gunicorn.

- **Lecciones aprendidas.** Hay que robustecer el cliente del LLM ante errores transitorios de la API (límites de cuota HTTP 429 y sobrecarga HTTP 503) con reintentos automáticos, en vez de asumir que cada llamada será exitosa. Separar el núcleo (LLM, MCP, log) de las interfaces y de los transportes permitió agregar la versión remota del servidor y la interfaz web sin tocar el cliente MCP ni el cliente del LLM. Y validar el protocolo a nivel de bytes ayuda a entender qué es MCP realmente: JSON-RPC 2.0 sobre un transporte.
- **Limitaciones y trabajo futuro.** El transporte Streamable HTTP está simplificado (sin SSE, sin cierre explícito de sesión, sin validación de `MCP-Protocol-Version`) y el servidor remoto no tiene autenticación, por lo que solo es adecuado para una demostración. El catálogo y las sesiones están en memoria: se pierden al reiniciar el servicio, y el cliente no reinicializa la sesión automáticamente cuando el servidor la rechaza. La interfaz web es de un solo usuario. La cuota gratuita de Gemini limita la cantidad de pruebas. Por último, las recomendaciones de la farmacia son ilustrativas y no sustituyen la consulta con un profesional de la salud. Como mejoras, se podría persistir el catálogo en una base de datos, agregar autenticación al endpoint remoto y soportar SSE.
- **Comentario final.** El proyecto permitió pasar de usar herramientas MCP como usuario a entender el protocolo por dentro: implementar el cliente y el servidor a mano, llevarlo a la nube y observarlo en la red conecta los conceptos de las capas del curso (enlace, red, transporte y aplicación) con un caso de uso real de la industria.